باسمه تعالی

مهدی رحیمی 40130112047 گزارش تمرین سری 5 بینایی ماشین

موضوع پروژه:

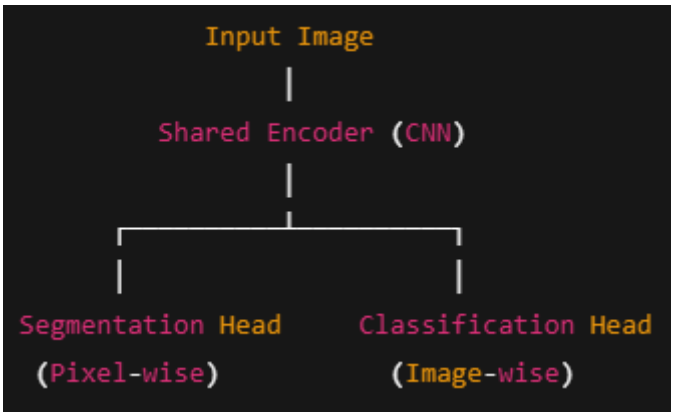
سگمنت کردن تصویر ورودی و تشخیص نوع حیوان (سگ یا گربه)

کد اصلی در فایل main.py هستش و در گرافیک هنگام اجرای کد باید گزینه ی Test و عکس مورد نظر حیوان برای تست انتخاب شود.

نصب کتابخانه ها:

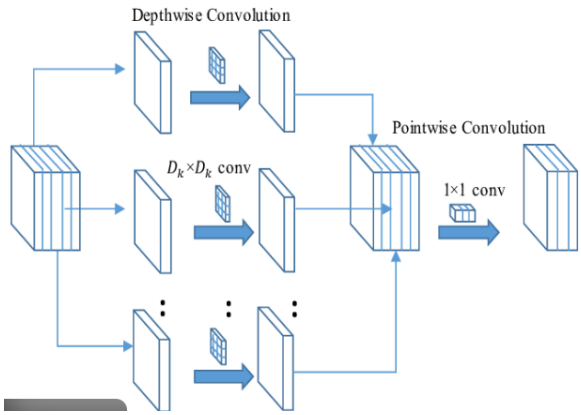
```
pip install numpy
pip install tensorflow
pip install tensorflow-datasets
pip install matplotlib
```

ایده کلی شبکه:



در کد من در ابتدا عکس ها به کمک کتابخانه ی keras با ابعاد 128 در 128 به عنوان ورودی وارد می‌شوند. این عکس ها به Encoder تغذیه می‌شوند. Encoder ها بخشی از شبکه هستند که لبه ها-بافت ها-شکل کلی حیوان و تفاوت حیوان با پس زمینه را یاد میگیرند. Encoder مورد استفاده در کد من از نوع MobileNetV2 است.

MobileNetV2 نوعی از معماری شبکه عصبی CNN است که سبک تر و سریع تر آن است. این معماری از تصویر خام ویژگی های مهم را استخراج میکند.



در این معماری در مرحله ی اول هرکانال تصویر به طور جداگانه پردازش می‌شود و فقط الگوهای مکانی(لبه-بافت) را یاد میگیرد. در مرحله ی بعد کانال ها با هم ترکیب می‌شوند و مفهوم کلی تصویر ساخته می‌شود. در نتیجه محاسبات خیلی کمتر و سرعت بالاتری نسبت به CNN خواهیم داشت.

لایه هایی که تعریف شدند به این صورت هستند:

```
layer_names = [
  "block_1_expand_relu",   # 64x64
  "block_3_expand_relu",   # 32x32
  "block_6_expand_relu",   # 16x16
  "block_13_expand_relu",  # 8x8
  "block_16_project",      # 4x4
]
```

هر کدام چه چیزی را نشان می‌دهند:

اندازه	معنی
64×64	لبه‌ها، مرزها
32×32	بافت مو
16×16	بخش‌های بدن
8×8	شکل کلی حیوان
4×4	مفهوم «حیوان بودن»

ما بعد از Encoder دو head برای پردازش تصویر داریم:

هد اول : Classification

در این هد برای تشخیص سگ یا گربه بودن ما داریم:

```
cls = GlobalAveragePooling2D()(x)
cls = Dense(128, relu)
cls_logits = Dense(2)
```

که GlobalAveragePoolig تمام کانال های ویژگی 4x4 را به یک بردار خلاصه میکند تا مدل بفهمد در کل تصویر چه ویژگی هایی غالب است. دو لایه ی بعدی لایه های عمیق هستند که برای ترکیب ویژگی ها برای تصمیم گیری می‌باشند.

هد دوم : Segmentation

برای این هد مدل میگوید که هر پیکسل برای حیوان است یا پس زمینه. ما در این هد Decoder داریم برای اینکه feature map فشرده شده را بگیرد و آن را مرحله به مرحله بزرگ کند تا به اندازه‌ی تصویر اولیه برسد و برای هر پیکسل تصمیم بگیرد. دیکدر این مسیر را طی می‌کند:

```
4x4 → 8x8 → 16x16 → 32x32 → 64x64 → 128x128
```

هر فلش یعنی یادگیری جزئیات جدید. Decoder با این لایه ها بزرگ‌سازی را انجام می‌دهد:

```
up_stack = [
    upsample(512, 3),
    upsample(256, 3),
    upsample(128, 3),
    upsample(64, 3),
]
```

هر upsample شامل این موارد است:

- Conv2DTranspose
- BatchNorm
- Relu

هر Conv2DTranspose یعنی :

یاد بگیر تصویر کوچک را چطور بزرگ کنی.

در آخرین لایه ی Decoder ما برای هر پیکسل 3 عدد خروجی می‌دهد مثلا:

$$[0.85, 0.10, 0.05]$$

یعنی : کلاس 0 (حیوان) – کلاس 1 (پس زمینه) – کلاس 2 (مرز)

که به کمک argmax ماسک پیکسلی ساخته می‌شود